SonarQube Integration

Static code analysis setup for the Meridian (myblogs) project

1. What is SonarQube?

SonarQube is a self-hosted static analysis platform that scans source code and reports on code quality and security. It does not run the application or execute tests itself — it parses source (and ingests test coverage reports you provide) and evaluates the result against a configurable set of rules and thresholds called a **Quality Gate**.

It checks four categories of issues:

- **Bugs** — code likely to behave incorrectly or crash (Reliability).
- **Vulnerabilities** — exploitable security flaws such as injection, XSS, hardcoded secrets, insecure crypto (Security). Found via static dataflow/taint analysis, not just pattern matching.
- **Code Smells** — maintainability issues: dead code, high complexity, poor structure.
- **Security Hotspots** — security-sensitive code (crypto, cookies, regex, deserialization) that needs a human to confirm whether it's actually exploitable in context, rather than being flagged outright as a vulnerability.

Alongside issues, it also computes non-issue metrics that factor into the gate: **test coverage** (from lcov/Cobertura reports you feed in) and **duplicated code density**.

> *Note: SonarQube Community Edition (what this project uses) does not scan third-party dependencies for known CVEs (no Software Composition Analysis) — it only analyzes your own source code.*

2. How This Repo Integrates SonarQube

The goal of this setup was to have exactly **one** way of invoking a scan and one way of checking the gate, used identically by a developer's machine, the IDE, and CI — rather than CI running different logic than what a developer can reproduce locally.

2.1 Project-level scan configuration — sonar-project.properties

Lives at the repo root and defines what gets scanned, independent of secrets:

- `sonar.sources` — scan root (`.`)
- `sonar.exclusions` — keeps node_modules/, dist/, *.db, uploads/, coverage/, .github/ out of analysis
- `sonar.javascript.file.suffixes` / `sonar.typescript.file.suffixes` — includes .vue files as JS/TS-analyzable
- `sonar.javascript.lcov.reportPaths` — points at each service's `coverage/lcov.info` so test coverage feeds into the gate
- `sonar.python.coverage.reportPaths` — points at `coverage-python.xml` so the meridian_agents test suite's coverage feeds into the gate too (see Section 6)

> *Note: sonar.projectKey=${SONAR_PROJECT_KEY} in this file is a literal placeholder — SonarScanner only substitutes environment variables written as ${env.VAR_NAME}. The real project key is supplied explicitly on the command line instead (see 2.3), so this placeholder has no functional effect.*

2.2 Secrets — .env (local) / GitHub Secrets (CI)

Three values are kept out of the properties file and out of git:

```
SONAR_HOST_URL=https://sonarqube.kube8t.com
SONAR_TOKEN=sqp_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
SONAR_PROJECT_KEY=base-kiwi-sauce
```

Locally these live in `.env` (gitignored) and are loaded by `dotenv-cli`, the same tool already used by the project's `start` / `start:backend` scripts. In CI they come from GitHub Actions repository secrets instead — dotenv-cli silently no-ops when no .env file is present, so the same npm command works unchanged in both places.

2.3 npm scripts — package.json

The scanner itself is the `@sonar/scan` npm package (installed as a root devDependency), exposing a `sonar-scanner` binary — no separate Java install or Docker image required.

```
"scripts": {
  "sonar":      "dotenv -e .env -- sh -c 'sonar-scanner \
            -Dsonar.projectKey="$SONAR_PROJECT_KEY" \
            -Dsonar.scanner.socketTimeout=300 \
            -Dsonar.scanner.responseTimeout=300'",
  "sonar:gate":  "dotenv -e .env -- sh scripts/sonar-quality-gate.sh",
  "sonar:check":  "npm run sonar && npm run sonar:gate"
}
```

The scan command is wrapped in an inner `sh -c '...'` deliberately: if `$SONAR_PROJECT_KEY` were expanded directly in the script string, npm's own shell would expand it before dotenv-cli has injected the variable from .env, resulting in an empty value. The inner shell defers expansion until after the environment is populated.

2.4 Quality Gate check script — scripts/sonar-quality-gate.sh

A standalone shell script (not inlined in package.json or the workflow file) so both local runs and CI call the exact same logic. See Section 5 for what it does.

2.5 IDE feedback — SonarQube for IDE (SonarLint)

As a complementary, faster feedback loop, the VS Code extension **SonarQube for IDE** can be connected to the same server (Connected Mode) so a developer sees issues inline while typing, using the same quality profile as the server. This is local-only analysis, though — it does not upload results or evaluate the Quality Gate. It is a supplement to the scan-and-gate flow, not a replacement for it.

3. Triggering a Scan Locally

From the repo root, with .env populated with real SONAR_* values:

```
npm run sonar        # run the scan only, uploads results to the server
npm run sonar:gate   # check the gate result of the most recent scan
npm run sonar:check  # run both in sequence; exits non-zero if the gate fails
```

Test coverage should be generated before scanning if the coverage condition matters — either per service (`npm run test:cov --prefix <service>`, plus `npm run coverage:python` for meridian_agents) or all at once via `npm run coverage:all` — since the properties file only picks up coverage that already exists on disk at scan time. `npm run sonar:full` chains `coverage:all` and `sonar:check` into a single command that reproduces the full CI pipeline locally.

4. Triggering from the CI Pipeline

Defined in `.github/workflows/sonarqube.yml`, as a job alongside the existing build-check job that runs on every pull request into main. As of this document, the job is present but **commented out** — it does not currently run or gate PRs.

```
sonarqube:
  steps:
    - actions/checkout@v4  (fetch-depth: 0)
    - actions/setup-node@v4
    - run: npm ci --ignore-scripts
    - run: npm run coverage:all   # 5 services + Python
    - name: SonarQube Scan
      env: {SONAR_TOKEN, SONAR_HOST_URL, SONAR_PROJECT_KEY}  # from GitHub Secrets
      run: npm run sonar
    - name: SonarQube Quality Gate check
      env: {SONAR_TOKEN, SONAR_HOST_URL, SONAR_PROJECT_KEY}
      run: npm run sonar:gate
```

The pipeline calls the identical npm scripts used locally — nothing CI-specific is duplicated. To enable it: uncomment the job, and add SONAR_TOKEN, SONAR_HOST_URL, and SONAR_PROJECT_KEY as repository secrets in GitHub. Once enabled, this becomes the mechanism that can actually block a PR merge on a failed gate (via a required status check), which running from a developer's machine or the IDE alone cannot enforce.

5. How the Quality Gate Check Works

Analysis and gate evaluation are asynchronous and happen in two stages:

- **Scan (client-side):** sonar-scanner analyzes the source locally and writes an intermediate report (protobuf files) under `.scannerwork/`. This is zipped and uploaded to the server via `POST /api/ce/submit`. The server responds with a task receipt, written locally to `.scannerwork/report-task.txt` (contains ceTaskId, dashboardUrl).
- **Processing (server-side):** the upload is queued as a Compute Engine (CE) background task. It ingests the report, computes final measures, matches issues against the active quality profile, and evaluates the Quality Gate conditions — all before the task reports as SUCCESS.

scripts/sonar-quality-gate.sh then does the following:

- Reads `ceTaskId` from report-task.txt.
- Polls `GET /api/ce/task?id=...` every 10 seconds (up to 30 attempts) until the task reaches SUCCESS, FAILED, or CANCELED.
- If the task did not reach SUCCESS, fails immediately — the gate result would otherwise reflect a stale prior analysis, not this run.
- Queries `GET /api/qualitygates/project_status?projectKey=...`, which returns the gate result already computed during processing (this call reads a result, it does not trigger new evaluation).
- Prints the overall status (OK or ERROR — SonarQube's API uses ERROR for a failed gate, not FAIL) and, on failure, the specific failing conditions with their actual value and threshold.
- Exits non-zero on anything other than OK, which is what allows CI to fail the build / block the PR on this step.

Example: a real gate failure observed on this project

Default gate conditions evaluate only *new* code (i.e. lines changed since the gate's baseline — the "leak period", currently the previous version tag), not the whole codebase. This example is from a scan run after a large batch of test files was added in a single session, so "new code" spans everything changed since that baseline, not just the most recent commit:

Metric	Actual	Required	Meaning
new_coverage	19.0%	≥ 100%	Coverage on lines changed since baseline
new_duplicated_lines_density	5.27%	≤ 3%	Too much duplicated new code
new_security_hotspots_reviewed	0.0%	= 100%	Hotspot(s) not yet manually reviewed
new_violations	73	= 0	New bugs / vulnerabilities / code smells introduced

A failing gate does not mean the scan itself failed — the scan always succeeds if it can reach the server and upload results. "Passing" or "failing" is purely the gate's evaluation of those results against its configured thresholds.

> *Note: the new_coverage threshold shown here (≥100%) is unusually strict compared to SonarQube's default "Sonar way" gate (typically ≥80%). It reflects this project's currently configured gate, which is worth reviewing in the SonarQube UI if the intent was the standard threshold. The other three conditions (violations, duplication, hotspot review) are independent of test coverage entirely — see Section 6 for coverage specifically.*

6. Code Coverage

6.1 What is code coverage?

Code coverage is a runtime metric, not a static-analysis one: as the test suite executes, a coverage tool instruments the code and records which lines (and optionally branches/functions) actually ran. The result is a percentage — e.g. "83% line coverage" means 83% of executable statements in the measured files were hit by at least one test.

It measures only whether a line *executed*, not whether the test that executed it asserted anything meaningful. A test that calls a function and checks nothing about its result still counts as "covering" that function. High coverage is necessary but not sufficient for a trustworthy test suite; low coverage, on the other hand, is a reliable signal that some code paths are completely unverified.

6.2 Why it matters here

- Untested paths are exactly where regressions hide silently — a suite that passes 100% of its own tests can still miss a broken function nobody wrote a test for.
- The Quality Gate's coverage condition is scoped to new/changed code specifically so it acts as a forcing function at the cheapest possible point: while the author still has full context on what the code is supposed to do, not months later during a bug report.
- It is a lagging indicator to monitor, not a target to game — this project's approach was to add real behavioral tests (including several that caught genuine production bugs along the way, e.g. a dependency version mismatch in api-gateway and a broken clipboard reference in BlogPost.vue) rather than shallow tests written purely to move the coverage number.

6.3 How coverage feeds into SonarQube

SonarQube does not execute tests or instrument code itself — it is purely a consumer of coverage reports produced by whatever test runner the project already uses. The report format is language-specific and declared in sonar-project.properties (Section 2.1):

- JS / TS / Vue → `sonar.javascript.lcov.reportPaths` — one `lcov.info` file per service, comma-separated (LCOV format).
- Python → `sonar.python.coverage.reportPaths` — a single `coverage-python.xml` (Cobertura XML format).

Both formats are, at their core, a list of covered/uncovered line numbers per source file — a standard interchange format that most coverage tools across ecosystems know how to both emit and consume. During analysis, the scanner cross-references each report's line numbers against the files it just parsed, and derives per-file, per-directory, whole-project, and git-diff-scoped (`new_coverage`) percentages — the last of which is what the Quality Gate actually evaluates.

> *Coverage reports must already exist on disk before npm run sonar runs — the scan is a point-in-time read of whatever coverage/lcov.info and coverage-python.xml happen to contain at that moment. Stale or missing reports silently produce 0% or a previous run's numbers, not an error.*

6.4 How it's configured in this repo

Coverage is generated per codebase using each ecosystem's native tooling, then every report path is declared centrally in sonar-project.properties so the scanner picks all of them up in one pass:

Codebase	Runner	Command	Report
auth-service, blog-service, media-service, api-gateway	Jest (ts-jest)	npm run test:cov (jest --coverage)	coverage/lcov.info
frontend	Vitest (v8 provider)	npm run test:cov (vitest run --coverage)	coverage/lcov.info
meridian_agents (Python)	pytest + pytest-cov	npm run coverage:python (--cov-report=xml)	coverage-python.xml

Each Jest config sets `collectCoverageFrom` to include all .ts/.js source while excluding `main.ts` and `app.module.ts` (framework bootstrap files with no meaningful logic to test). Vitest's config similarly excludes `main.js`, `router/**`, `sw.js`, and the placeholder `HelloWorld.vue` component. On the Python side, `pyproject.toml`'s `[tool.coverage.run]` scopes collection to `source = ["meridian_agents"]` and omits `meridian_agents/tests/*` itself from being counted as coverable code.

`npm run coverage:all` chains all six coverage commands (five services + Python) in sequence, so a single command regenerates every report SonarQube needs before a scan. `npm run sonar:full` then runs `coverage:all`, the scan, and the gate check in order — the one command that reproduces the full CI pipeline end-to-end from a clean checkout.
